

PROJECT ARGUS

Privileged Access Misuse & Insider Threat Detection on AWS

Just-In-Time Access • Continuous Monitoring • Identity-Aware Risk Engine • Automated Response

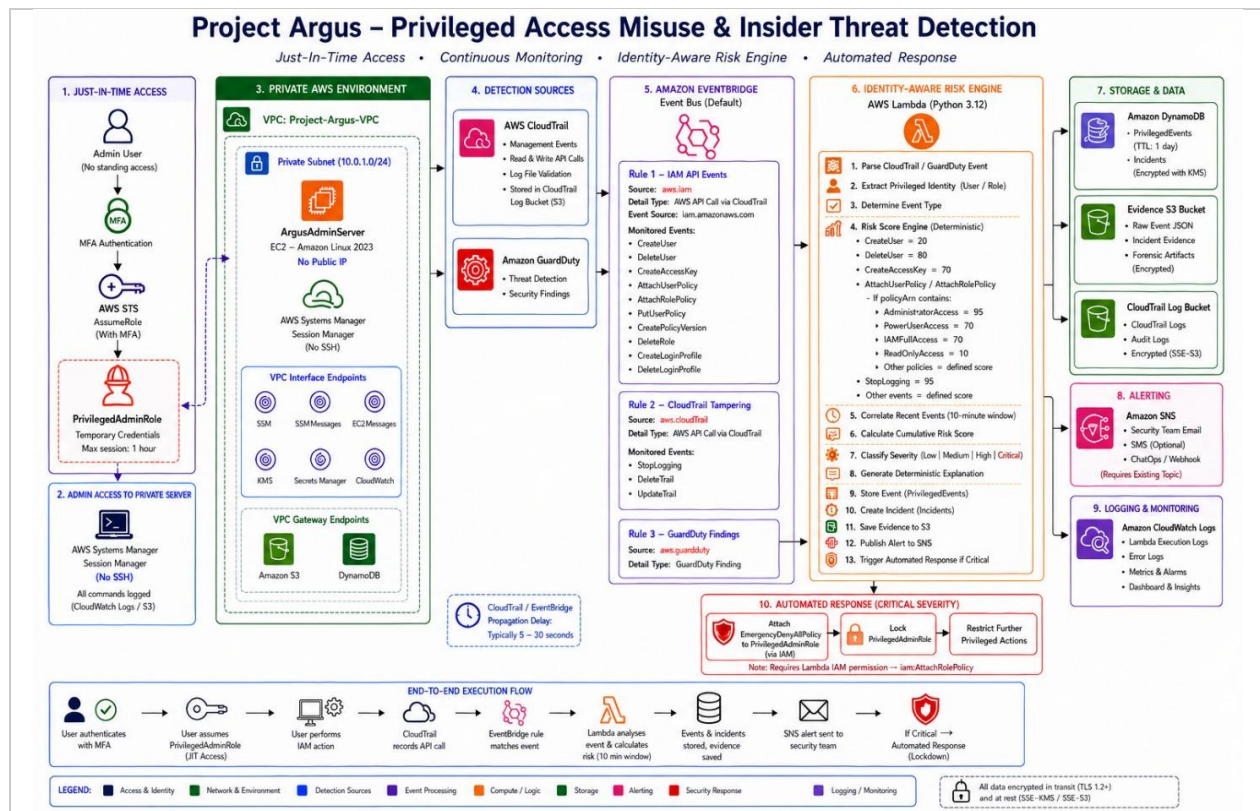


Figure 0.1 — Project Argus end-to-end architecture

1. Executive Summary

Banks and large organizations grant certain employees “privileged access” — the ability to create users, change permissions, delete accounts, disable logging, and more. This access is necessary for administrators to do their jobs, but it is also the single biggest risk in the system: if misused, by a malicious insider, a negligent employee, or an attacker who has stolen those credentials, the damage can be enormous, because the perpetrator is acting as a trusted identity rather than breaking in from outside.

Traditional security tools such as firewalls and intrusion detection systems are built to catch outsiders crossing a boundary. They are largely ineffective here because there is no boundary being crossed — the actor already holds valid credentials and is doing things the system is designed to let them do. Privileged access misuse therefore needs a fundamentally different approach: not “block bad traffic,” but make power temporary, watch what that power is used for, and respond fast when it looks wrong.

Project Argus is a fully serverless AWS system that implements exactly that approach. It combines Just-In-Time privileged access, dual-source behavioral detection (CloudTrail and GuardDuty), an identity-aware risk-scoring Lambda, and automatic containment of compromised sessions — all without any standing administrative access anywhere in the account.

1.1 Core Design Philosophy

Four principles drive every design decision in the system, in order of precedence:

1. Nobody should have standing power. Access is requested, proven, and temporary — never permanently granted.
2. Watch behavior, not single events. One action can be innocent; a sequence of specific actions in a short window is often not.
3. Explain findings in plain language. A raw risk score means nothing to a human who has to decide what to do next.
4. Respond automatically for the worst cases. Waiting for a human to notice and act gives an attacker more time to do damage.

2. System Architecture

The diagram below shows the complete data and control flow of Project Argus, from the moment an administrator authenticates through to automated containment and alerting.

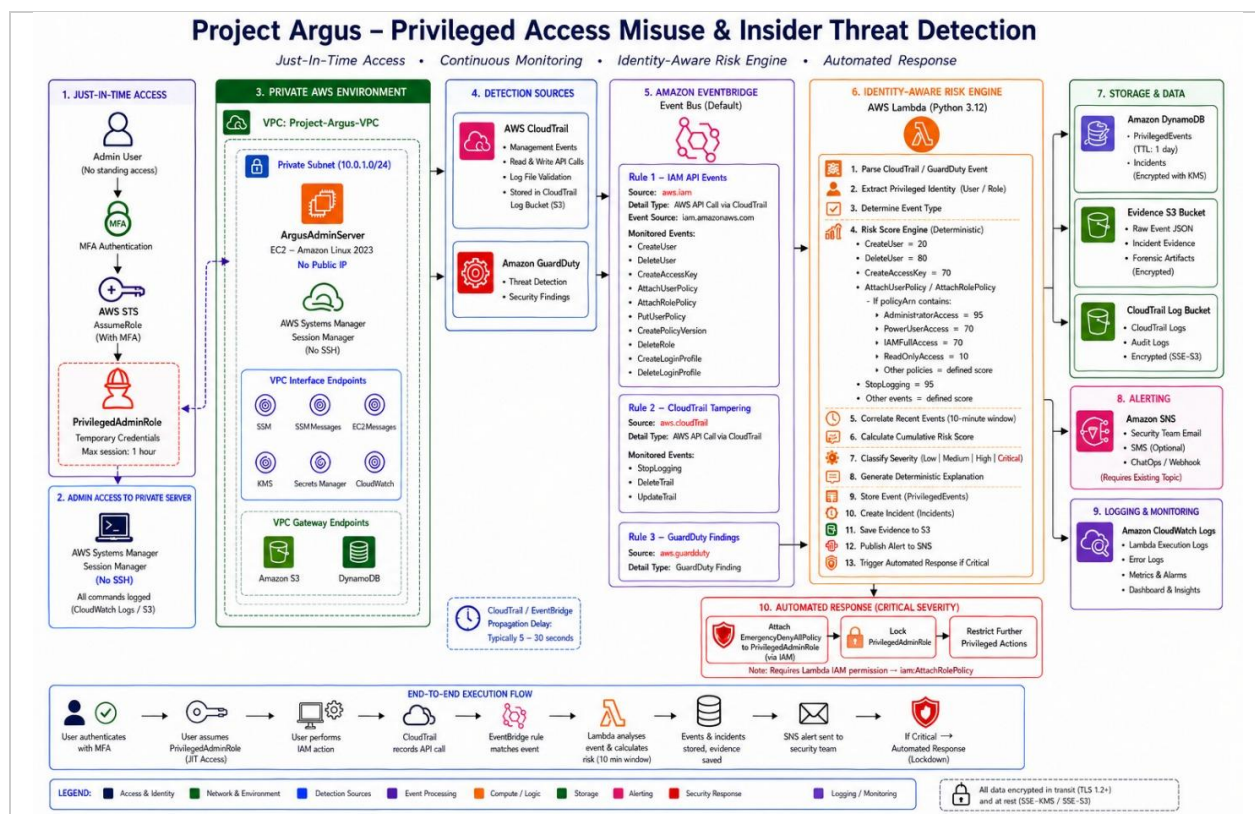


Figure 2.1 — Project Argus full architecture: identity, network, detection, routing, risk engine, storage, and alerting

The remainder of this document walks through every stage of this diagram in the order events actually flow through the system, using live screenshots captured directly from the deployed AWS account.

3. Identity — Nobody Has Power by Default

The system's first defense is structural: no identity carries privileged permissions at rest.

Design principle

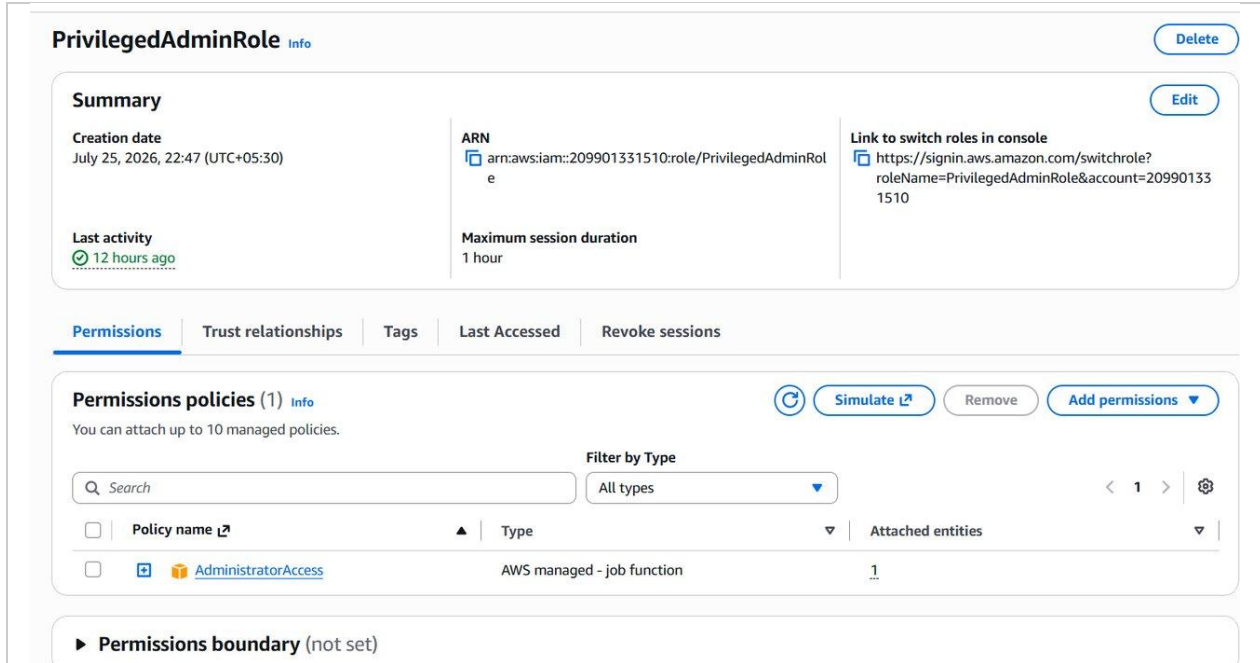
admin-test-user has zero attached permissions. To do anything privileged, it must explicitly assume a separate IAM role, PrivilegedAdminRole, using AWS STS. This is called Just-In-Time (JIT) access.

Two conditions make this escalation meaningful rather than a formality:

- MFA is required — the role's trust policy explicitly demands multi-factor authentication before the assumption is even permitted.
- The session expires — capped at one hour, so even a leaked credential only grants a small window of usable power.

Even a fully compromised admin-test-user account cannot cause damage without also defeating MFA — and even then, only for a limited time.

3.1 The privileged role and its trust policy



PrivilegedAdminRole [Info](#) [Delete](#)

Summary [Edit](#)

Creation date
July 25, 2026, 22:47 (UTC+05:30)

ARN
[arn:aws:iam::209901331510:role/PrivilegedAdminRole](#)

Link to switch roles in console
<https://signin.aws.amazon.com/switchrole?roleName=PrivilegedAdminRole&account=209901331510>

Last activity
12 hours ago

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (1) [Info](#) [Simulate](#) [Remove](#) [Add permissions](#)

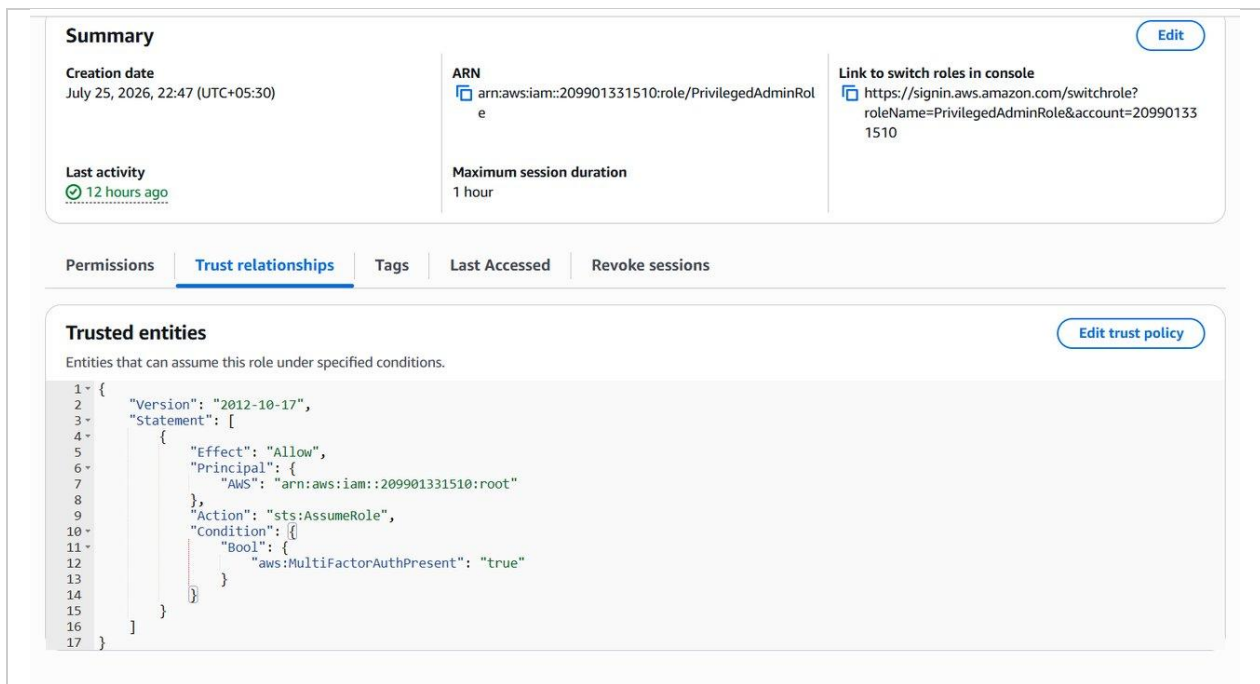
You can attach up to 10 managed policies.

Filter by Type
All types

Policy name	Type	Attached entities
AdministratorAccess	AWS managed - job function	1

► **Permissions boundary** (not set)

Figure 3.1 — PrivilegedAdminRole: maximum session duration capped at 1 hour



Summary [Edit](#)

Creation date
July 25, 2026, 22:47 (UTC+05:30)

ARN
[arn:aws:iam::209901331510:role/PrivilegedAdminRole](#)

Link to switch roles in console
<https://signin.aws.amazon.com/switchrole?roleName=PrivilegedAdminRole&account=209901331510>

Last activity
12 hours ago

Maximum session duration
1 hour

Permissions | **Trust relationships** | Tags | Last Accessed | Revoke sessions

Trusted entities [Edit trust policy](#)

Entities that can assume this role under specified conditions.

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": {
7         "AWS": "arn:aws:iam::209901331510:root"
8       },
9       "Action": "sts:AssumeRole",
10      "Condition": {
11        "Bool": {
12          "aws:MultiFactorAuthPresent": "true"
13        }
14      }
15    }
16  ]
17 }
```

Figure 3.2 — Trust policy requiring `aws:MultiFactorAuthPresent = true` before `sts:AssumeRole` is allowed

3.2 Assuming the role

An operator switches into the privileged role explicitly through the AWS console's Switch Role flow, naming the exact role and account rather than being logged in with standing power.

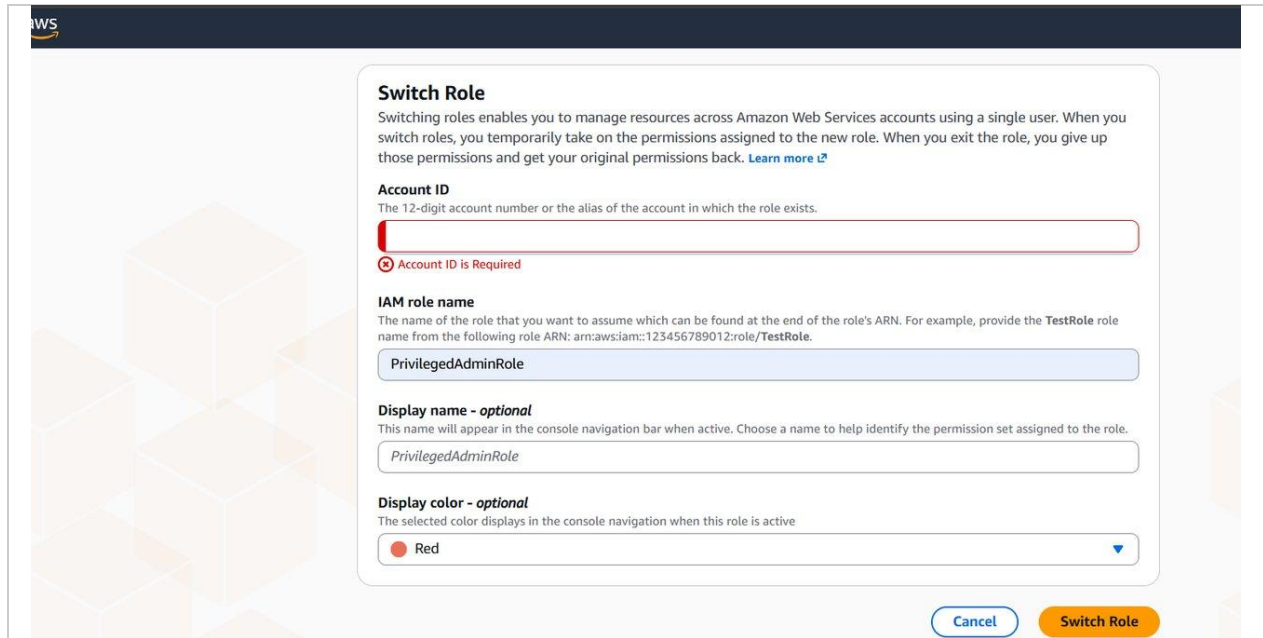


Figure 3.3 — Switching into PrivilegedAdminRole via the AWS console

3.3 Proof that the base identity has no standing access

Before assuming the role, the base user is denied access to ordinary console resources — confirming there is no residual privilege sitting on the identity itself.

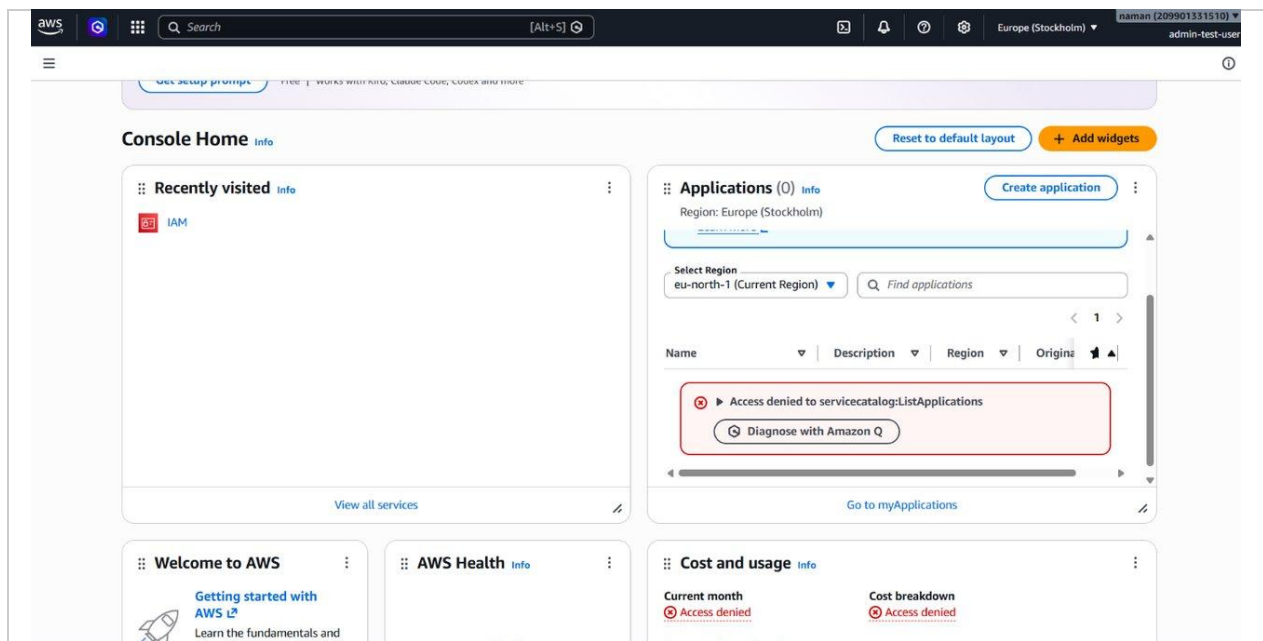


Figure 3.4 — admin-test-user denied access to Service Catalog and cost data without an assumed role

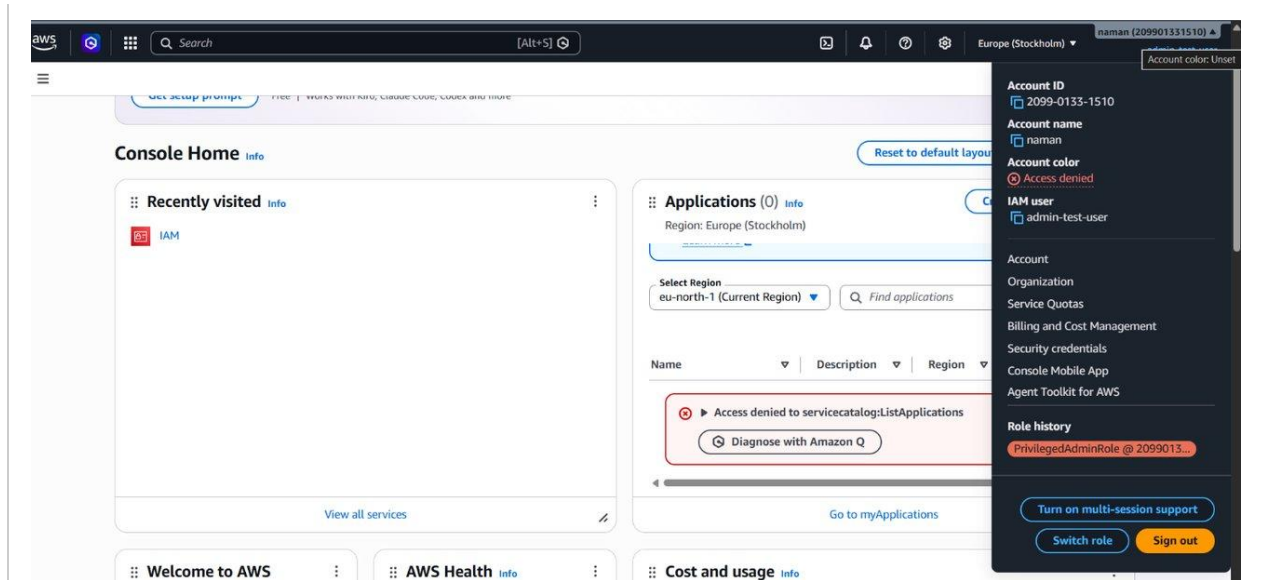


Figure 3.5 — Account panel confirming “Access denied” as the account color status for the base identity

4. The Private Network — Nowhere to Hide, Nowhere to Leak

A simulated banking admin server, ArgusAdminServer (an EC2 instance), sits inside a VPC with no Internet Gateway at all — not blocked, genuinely absent. It is reached only through AWS Systems Manager Session Manager, never SSH. This matters for two reasons: there is no SSH key that could be stolen or shared, and every command typed during a session is centrally logged and tied to the exact IAM identity that ran it.

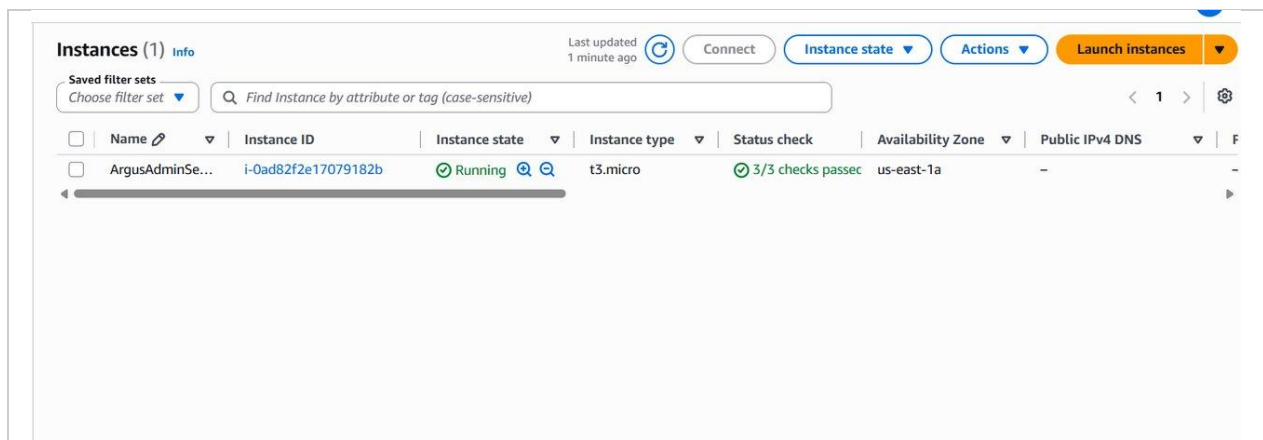


Figure 4.1 — ArgusAdminServer (t3.micro) running with passed status checks, no public IP

4.1 VPC, subnet, and routing

The screenshot displays the AWS Management Console interface. At the top, the 'Your VPCs' section shows a table with two VPCs: a default VPC and 'Argus-VPC'. Below this, the 'Resource map' for 'vpc-09b1c34eb78a9dc4c / Argus-VPC' is shown. The map includes four components: 'VPC' (Argus-VPC), 'Subnets (1)' (PrivateSubnet-1), 'Route tables (1)' (rtb-07b5850984eac46cd), and 'Network Connections (2)' (s3 and dynamodb). The PrivateSubnet-1 is connected to the route table, which is in turn connected to the S3 and DynamoDB network connections.

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR	IPv6 CIDR
-	vpc-04215cc94c21fa4b4	Available	-	-	Off	172.31.0.0/16	-
Argus-VPC	vpc-09b1c34eb78a9dc4c	Available	-	-	Off	10.0.0.0/16	-

Figure 4.2 — Argus-VPC resource map: one private subnet, one route table, connections to S3 and DynamoDB only

The screenshot shows the 'Subnets (1/1)' page with 'PrivateSubnet-1' selected. Below the table, the 'Details' section for 'subnet-04100ab494f254e8d / PrivateSubnet-1' is displayed. The details include Subnet ID, Subnet ARN, State (Available), IPv4 CIDR (10.0.1.0/24), Availability Zone (us-east-1), Network ACL, Auto-assign customer-owned IPv4 address (No), IPv6 CIDR reservations, Resource name DNS AAAA record (Disabled), Subnet ARN, Available IPv4 addresses (244), Network border group (us-east-1), Default subnet (No), Customer-owned IPv4 pool, IPv6-only (No), DNS64 (Disabled), State (Available), IPv6 CIDR, VPC (vpc-09b1c34eb78a9dc4c | Argus-VPC), Auto-assign public IPv4 address (No), Outpost ID, Hostname type (IP name), Owner (209901331510), Block Public Access (Off), IPv6 CIDR association ID, Route table (rtb-07b5850984eac46cd), Auto-assign IPv6 address (No), IPv4 CIDR reservations, and Resource name DNS A record (Disabled).

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR	IPv6 CIDR association ID
PrivateSubnet-1	subnet-04100ab494f254e8d	Available	vpc-09b1c34eb78a9dc4c Argus-VPC	Off	10.0.1.0/24	-	-

Figure 4.3 — PrivateSubnet-1 (10.0.1.0/24) with public IPv4 auto-assign disabled

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
argus-rt	rtb-07b5850984eac46cd	-	-	Yes	vpc-09b1c34eb78a9dc4c Argu...	209901331510
-	rtb-0e6ee75b5223d1d33	-	-	Yes	vpc-04215cc94c21fa4b4	209901331510

rtb-07b5850984eac46cd / argus-rt			
Details	Routes	Subnet associations	Edge associations
Route table ID rtb-07b5850984eac46cd VPC vpc-09b1c34eb78a9dc4c Argus-VPC	Main Yes Owner ID 209901331510	Explicit subnet associations -	Edge associations -

Figure 4.4 — Route table argus-rt with no route to an Internet Gateway

4.2 Security group

Security Groups (1/3)

info

Find security groups by attribute or tag

Actions

Export security groups to CSV

Create security group

<

1

>

	Name	Security group ID	Security group name	VPC ID	Description	Owner	Inbox
	-	sg-08aa0238264d00307	default	vpc-04215cc94c21fa4b4	default VPC security group	209901331510	1 Pen
	-	sg-0d5a506c16a04f49b	default	vpc-09b1c34eb78a9dc4c	default VPC security group	209901331510	1 Pen
	-	sg-061f1981a689e6359	AdminServerSG	vpc-09b1c34eb78a9dc4c	AdminServerSG	209901331510	1 Pen

sg-061f1981a689e6359 - AdminServerSG

Details

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (1)

Search

Figure 4.5 — AdminServerSG: the only inbound rule permits HTTPS (443) from the security group itself

4.3 VPC endpoints — precise, named exceptions

Since the subnet has no internet access, a set of VPC Endpoints give it exact, named reachability — SSM, SSM Messages, EC2 Messages, Secrets Manager, KMS, CloudWatch (Interface endpoints), plus S3 and DynamoDB (Gateway endpoints) — rather than opening broad internet access just so a couple of services are reachable.

The screenshot shows the 'Endpoints (8)' page in the AWS console. It contains a table with the following columns: Name, VPC endpoint ID, Endpoint type, Status, Service name, and Service network. All endpoints are in an 'Available' state.

Name	VPC endpoint ID	Endpoint type	Status	Service name	Service network
ssm	vpce-0f4585b6f57e77de3	Interface	Available	com.amazonaws.us-east-1.ssm	-
ssmmessages	vpce-02d14c7a02c351dca	Interface	Available	com.amazonaws.us-east-1.ssmmessages	-
ec2messages	vpce-0cbf90ffa049c4553	Interface	Available	com.amazonaws.us-east-1.ec2messages	-
secretsmanager	vpce-094413e6e004374e0	Interface	Available	com.amazonaws.us-east-1.secretsmanager	-
kms	vpce-06ba334c95fa05c60	Interface	Available	com.amazonaws.us-east-1.kms	-
monitoring	vpce-06827ba502b2fb4d	Interface	Available	com.amazonaws.us-east-1.monitoring	-
s3	vpce-0cf0719530c24f2f7	Gateway	Available	com.amazonaws.us-east-1.s3	-
dynamodb	vpce-0b6aa3ae3ef30be31	Gateway	Available	com.amazonaws.us-east-1.dynamodb	-

Figure 4.6 — All 8 VPC endpoints available: 6 interface endpoints and 2 gateway endpoints

5. Detection — Two Independent Sources Watching Everything

Every action taken anywhere in the AWS account is recorded by CloudTrail — a complete, factual, tamper-evident log of every API call. Separately, Amazon GuardDuty applies AWS's trained machine-learning models to spot anomalous patterns CloudTrail alone would not flag, such as unusual API sequences or compromised-credential indicators.

Using both matters because they cover different blind spots: CloudTrail catches anything you explicitly wrote detection rules for; GuardDuty catches things nobody thought to write a rule for.

The screenshot shows the 'PrivilegedAccessTrail' console page. It displays general details for a trail named 'PrivilegedAccessTrail'. The trail is logging, multi-region, and applies to the organization. Log file validation is enabled, and the last log file was delivered on July 28, 2026. SNS notification delivery is disabled.

General details	Trail log location	Log file validation	SNS notification delivery
Trail logging Logging Trail name PrivilegedAccessTrail Multi-region trail Yes Apply trail to my organization Not enabled	Trail log location aws-cloudtrail-logs-209901331510-8d7ee0f6/AWSLogs/209901331510 Last log file delivered July 28, 2026, 22:22:32 (UTC+05:30) Log file SSE-KMS encryption Not enabled	Log file validation Enabled Last file validation delivered July 28, 2026, 22:08:57 (UTC+05:30)	SNS notification delivery Disabled Last SNS notification -

Figure 5.1 — PrivilegedAccessTrail: multi-region trail, logging enabled, log file validation on

The screenshot shows the 'GuardDuty Summary' dashboard. It provides an overview of security trends, including the number of attack sequences, total findings, resources with findings, and accounts with findings. The dashboard also shows findings by severity and runtime monitoring coverage.

Overview	Total findings	Resources with findings	Accounts with findings
Attack sequences: 0 Findings: 1 Critical: 0 , High: 0 , Medium: 0 , Low: 1	1	1	1

Findings
 The API GetAccountColor was invoked using root credentials.
 Severity: Low

Findings by severity
 Total findings: 1

Runtime Monitoring coverage
 Monitor runtime activity to detect threats to your compute workloads on Amazon ECS (including AWS Fargate), Amazon EKS, and Amazon EC2.
 Enable Runtime Monitoring

Most common finding types
 PolicyIAMUser/RootCredentialUsage

Figure 5.2 — GuardDuty summary dashboard showing active findings and coverage

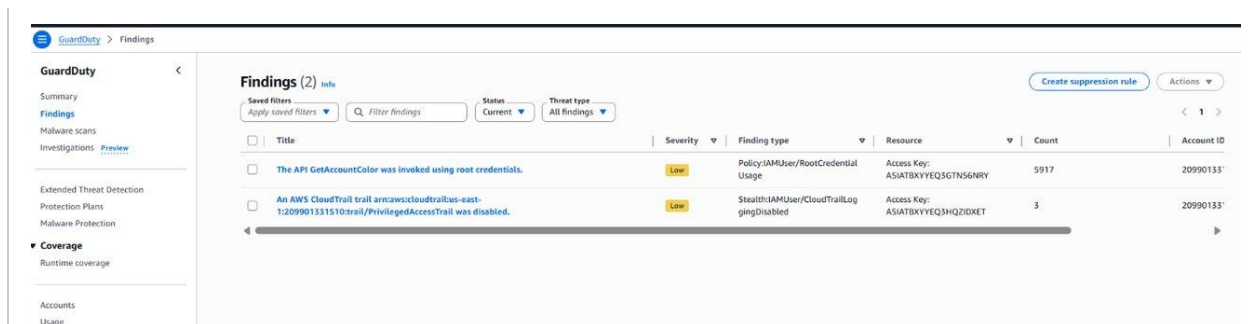


Figure 5.3 — GuardDuty findings: root-credential usage and a CloudTrail-logging-disabled finding, both flagged automatically

6. Routing — EventBridge Connects Detection to Decision-Making

Amazon EventBridge watches for specific patterns in CloudTrail and GuardDuty output and routes matching events to a single Lambda function. There are three separate rules, because different event types need to be filtered on different fields.

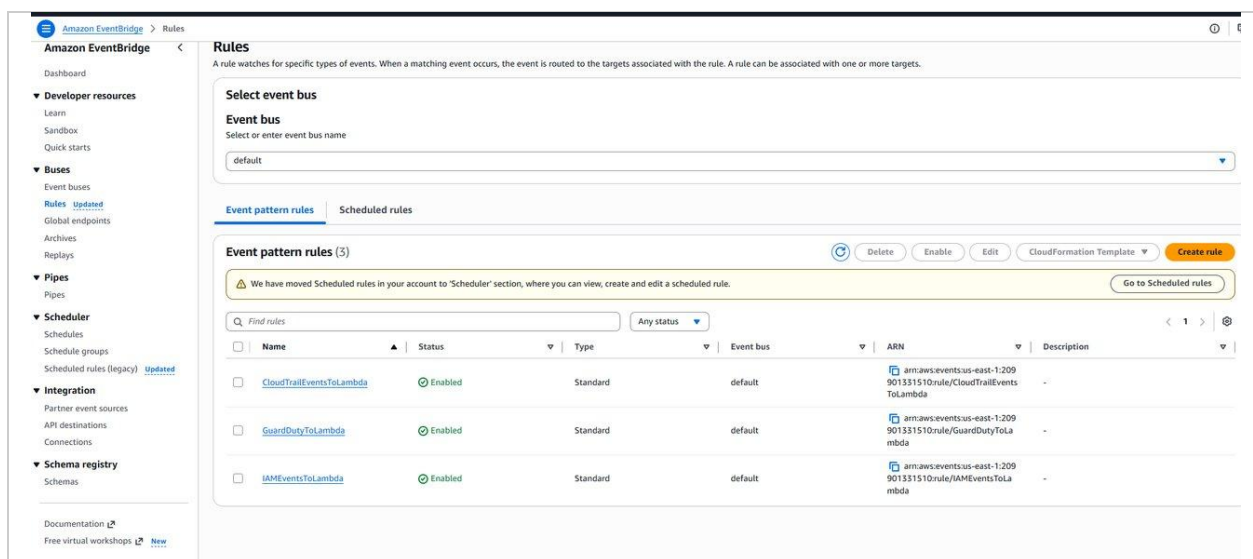


Figure 6.1 — Three enabled EventBridge rules, all targeting the RiskEngine Lambda

6.1 Rule 1 — IAM-related actions

Matches high-value identity actions: CreateUser, DeleteUser, AttachUserPolicy, AttachRolePolicy, CreateAccessKey, PutPolicy, and more.

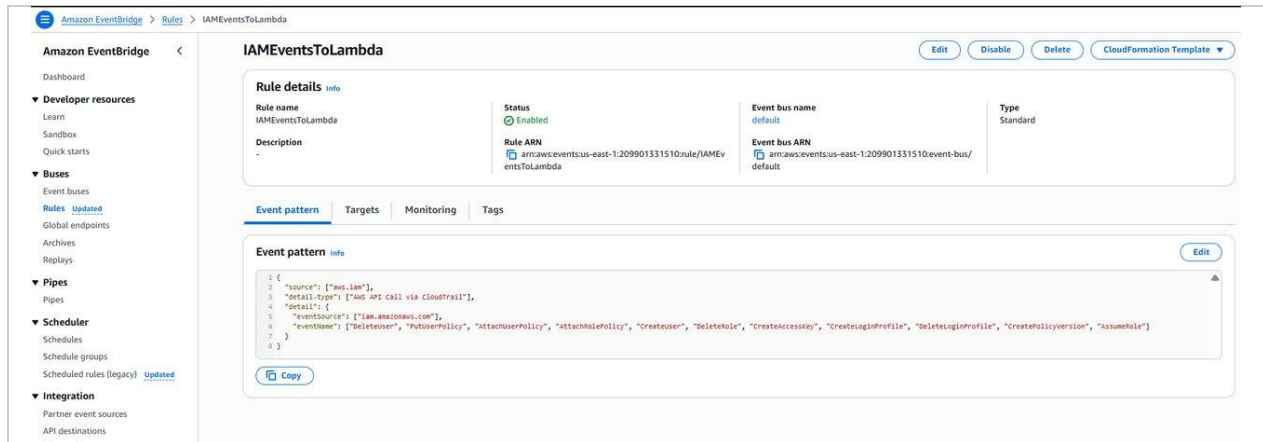


Figure 6.2 — IAMEventsToLambda event pattern

6.2 Rule 2 — CloudTrail's own actions

Matches StopLogging, DeleteTrail, and UpdateTrail — disabling your own audit trail is itself a huge red flag.

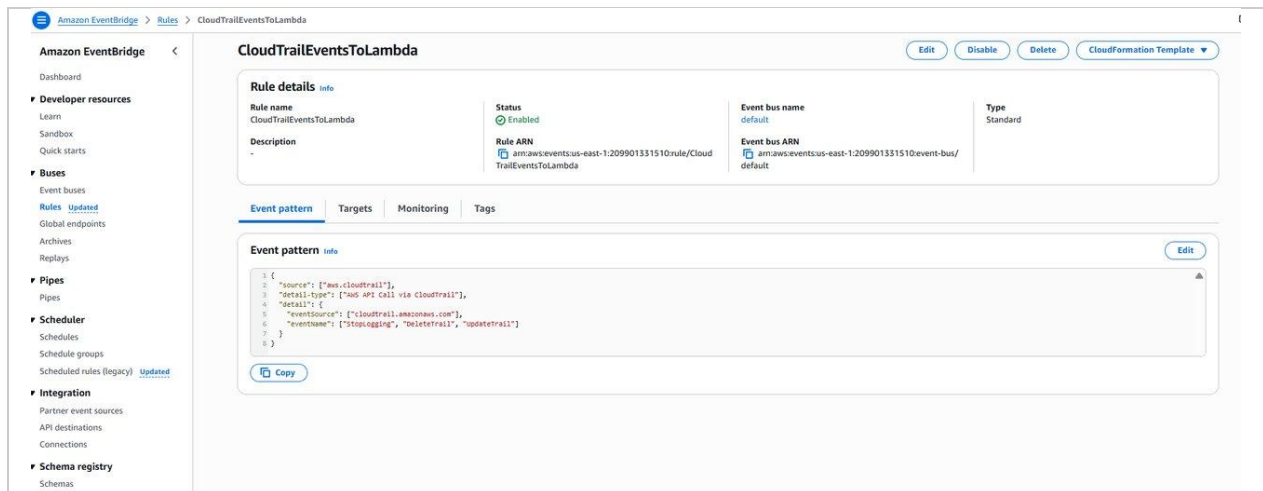


Figure 6.3 — CloudTrailEventsToLambda event pattern

6.3 Rule 3 — GuardDuty findings

Matches any GuardDuty finding, regardless of type, so ML-detected anomalies always reach the risk engine.

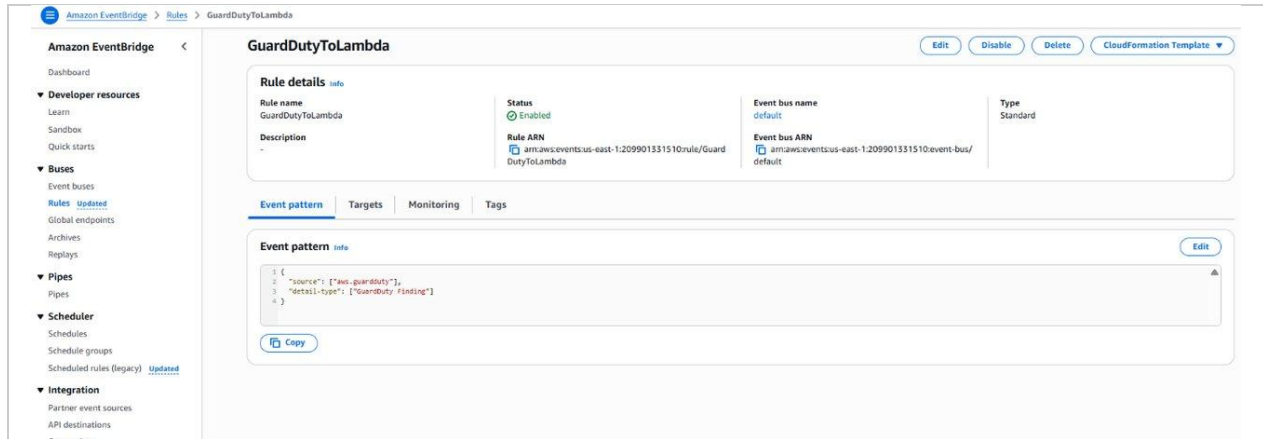


Figure 6.4 — GuardDutyToLambda event pattern

7. The Risk Engine — The Brain of the System

A single AWS Lambda function, RiskEngine, receives every routed event. Everything else in the system either feeds it data or carries out what it decides. On every invocation it performs eight steps:

5. Parses the incoming event to extract the identity responsible and the specific action taken.
6. Resolves assumed-role session identifiers back to a consistent role name, so repeated logins correlate correctly instead of looking like unrelated identities.
7. Scores the action — inspecting which specific policy was granted (e.g. AdministratorAccess vs. ReadOnlyAccess), not just that a policy-attach event occurred.
8. Correlates all of this identity's activity over the last 10 minutes, not just the single triggering event.
9. Classifies cumulative severity as Low, Medium, High, or Critical.
10. Generates a plain-language explanation of what happened and why it matters.
11. Stores the event, creates an incident record, and saves forensic evidence to S3.
12. On a Critical finding, automatically attaches a deny-all policy to contain the session.

7.1 The scoring model

Not every action is equally risky. Representative weights used by the engine:

CreateUser	20
DeleteUser	80
CreateAccessKey	70
AttachUserPolicy / AttachRolePolicy (AdministratorAccess)	95
AttachUserPolicy / AttachRolePolicy (PowerUserAccess)	70
AttachUserPolicy / AttachRolePolicy (IAMFullAccess)	70
AttachUserPolicy / AttachRolePolicy (ReadOnlyAccess)	10
StopLogging	95

A single CreateAccessKey call might be routine. But CreateUser → AttachUserPolicy(AdministratorAccess) → DeleteUser, all within minutes, adds up to something that looks like privilege escalation followed by covering tracks — and only correlation across the 10-minute window reveals that pattern.

7.2 Deterministic explanation, not an external AI call

The plain-language explanation is generated deterministically inside the Lambda rather than by calling out to an external AI service. This removes a dependency that could fail for reasons outside the system's control (billing, subscriptions, availability) while still delivering the same core benefit: a readable, specific explanation instead of a raw number.

7.3 Deny the role, not the user

The one decision most people get wrong

On a Critical finding, the system attaches an EmergencyDenyAllPolicy to the role itself, not the original IAM user. Once an identity has assumed a role, its permissions for that session are evaluated against the role's policies, not the user's — so a deny on the user would silently do nothing to an already-active session, while a deny on the role takes effect on the very next API call.

Section 9 of this document shows this control working against a live session, captured directly from the deployed system.

7.4 The function in the console

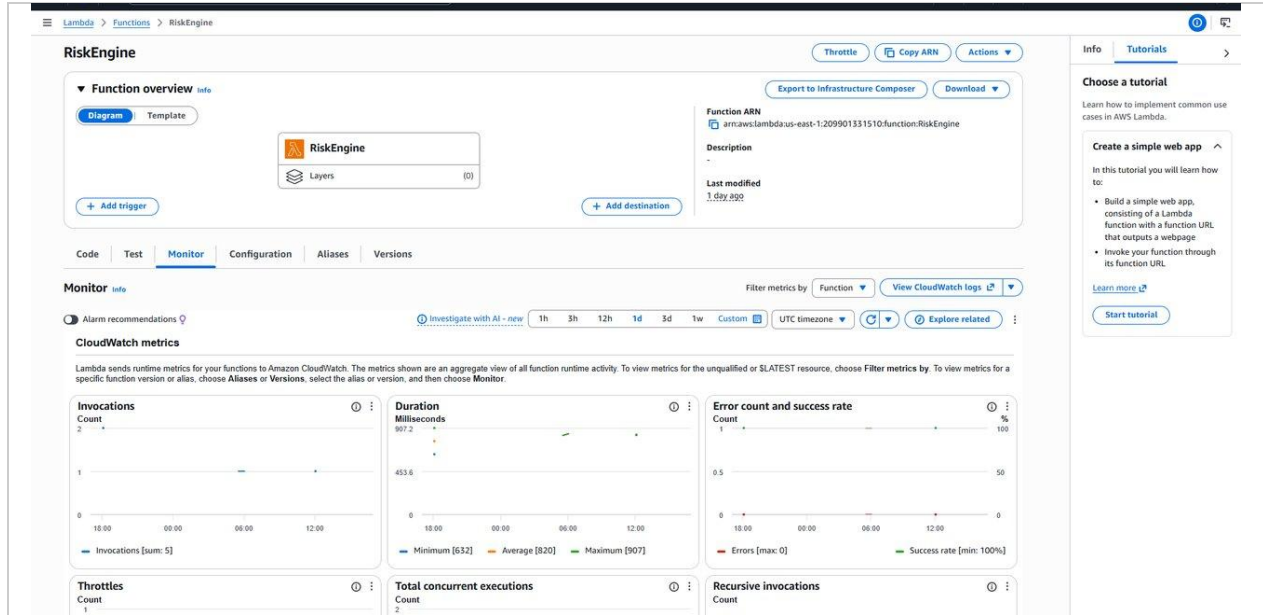


Figure 7.1 — RiskEngine Lambda: invocation, duration, and error-rate metrics

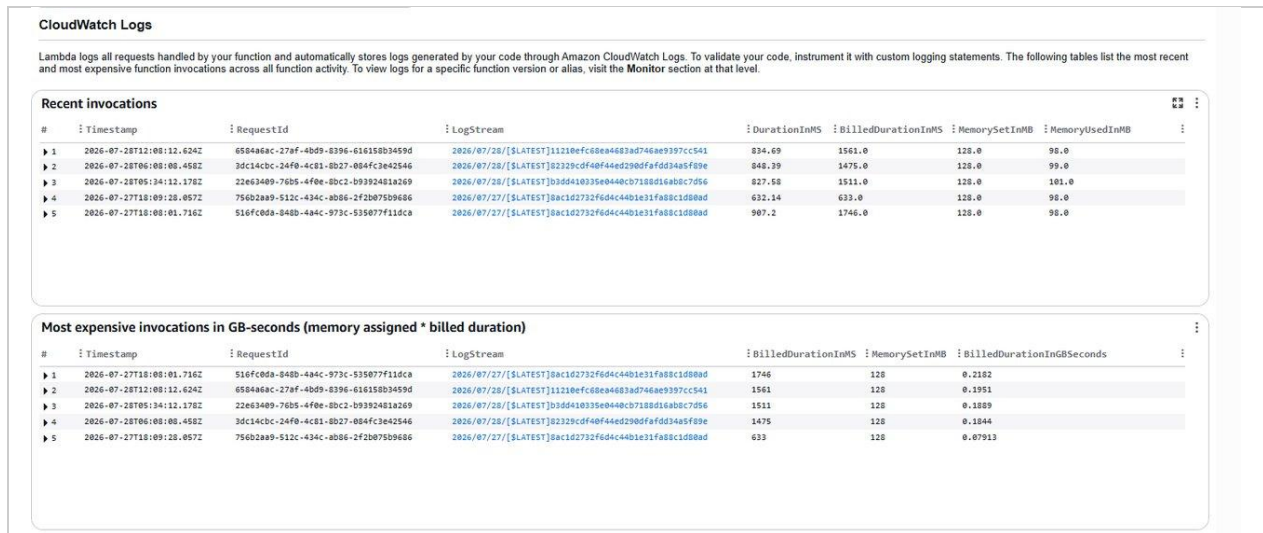


Figure 7.2 — Recent invocations and most expensive invocations, in GB-seconds

8. Storage and Evidence — Nothing Is Lost

Two DynamoDB tables back the system: a rolling window of recent activity that automatically expires entries after one day via TTL, so it never grows unbounded, and a permanent Incidents table kept as a durable audit trail.

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode	Total size	Table status
Incidents	Active	IncidentId (S)	-	0	0	OFF	☆	On-demand	On-demand	575.3 megabytes	Standalone
PrivilegedEvents	Active	userid (S)	timestamp (S)	0	0	OFF	☆	On-demand	On-demand	1.4 megabytes	Standalone

Figure 8.1 — PrivilegedEvents (rolling window, TTL enabled) and Incidents (permanent) tables

IncidentId (String)	automatedResponse	events	eventType	explanation	policyArn	severity	snsError	status	timestamp	totalScore
Root-1785175681	None	[{"M": {"%...}}	GuardDuty...	User 'Root' tr...	<empty>	High		OPEN	2026-07-2...	70
Root-1785218887	None	[{"M": {"%...}}	GuardDuty...	User 'Root' tr...	<empty>	High		OPEN	2026-07-2...	70

Figure 8.2 — Incidents table: each row carries severity, explanation, automated response, and total score

userid (String)	timestamp (String)	eventId	eventSource	eventType	expireAt	policyArn	score	sourceIp
unknown-user	2026-07-27T04:35:02...	unknown	unknown	UnknownEv...	1785213302	<empty>	10	unknown
unknown-user	2026-07-27T04:35:02...	unknown	unknown	UnknownEv...	1785213302	<empty>	10	unknown
unknown-user	2026-07-27T05:39:40...	6f60046e-7...	events.amazo...	PutRule	1785217180	<empty>	10	104.28.157...
unknown-user	2026-07-27T05:39:42...	2899915c-...	sts.amazonaw...	AssumeRole	1785217182	<empty>	20	events.ama...
unknown-user	2026-07-27T05:39:44...	f1ad8b90-c...	sts.amazonaw...	AssumeRole	1785217184	<empty>	20	events.ama...
unknown-user	2026-07-27T05:39:47...	40f7443d-f...	sts.amazonaw...	AssumeRole	1785217187	<empty>	20	events.ama...
unknown-user	2026-07-27T05:39:49...	4af540d3-5...	sts.amazonaw...	AssumeRole	1785217189	<empty>	20	events.ama...
unknown-user	2026-07-27T05:39:51...	5515d3bf-...	sts.amazonaw...	AssumeRole	1785217191	<empty>	20	lambda.am...
unknown-user	2026-07-27T05:39:52...	a2a918b7-f...	sts.amazonaw...	AssumeRole	1785217192	<empty>	20	events.ama...

Figure 8.3 — PrivilegedEvents table: every scored event in the rolling correlation window, with per-event score and TTL expiry

Every High or Critical incident is also written as a JSON file to S3 for forensic evidence, and the raw CloudTrail log stream itself is preserved separately.

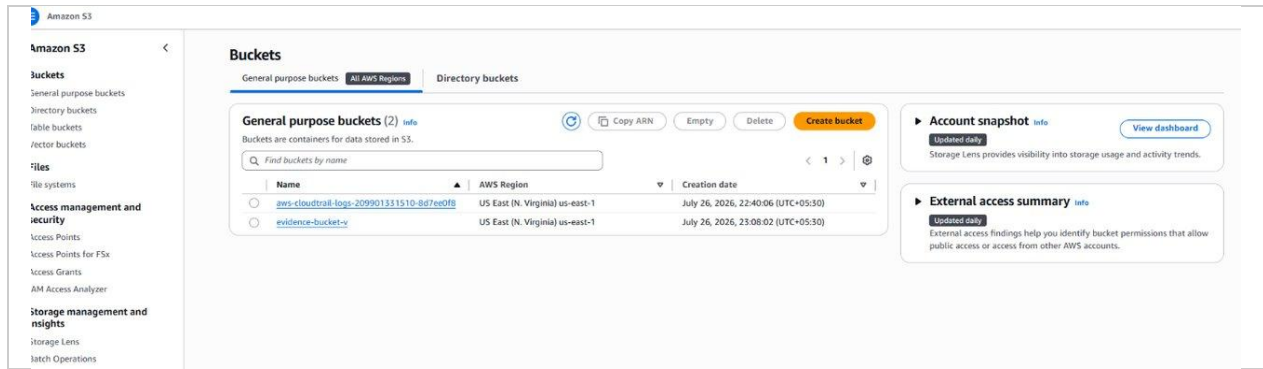


Figure 8.4 — Two S3 buckets: raw CloudTrail logs and the incident evidence bucket

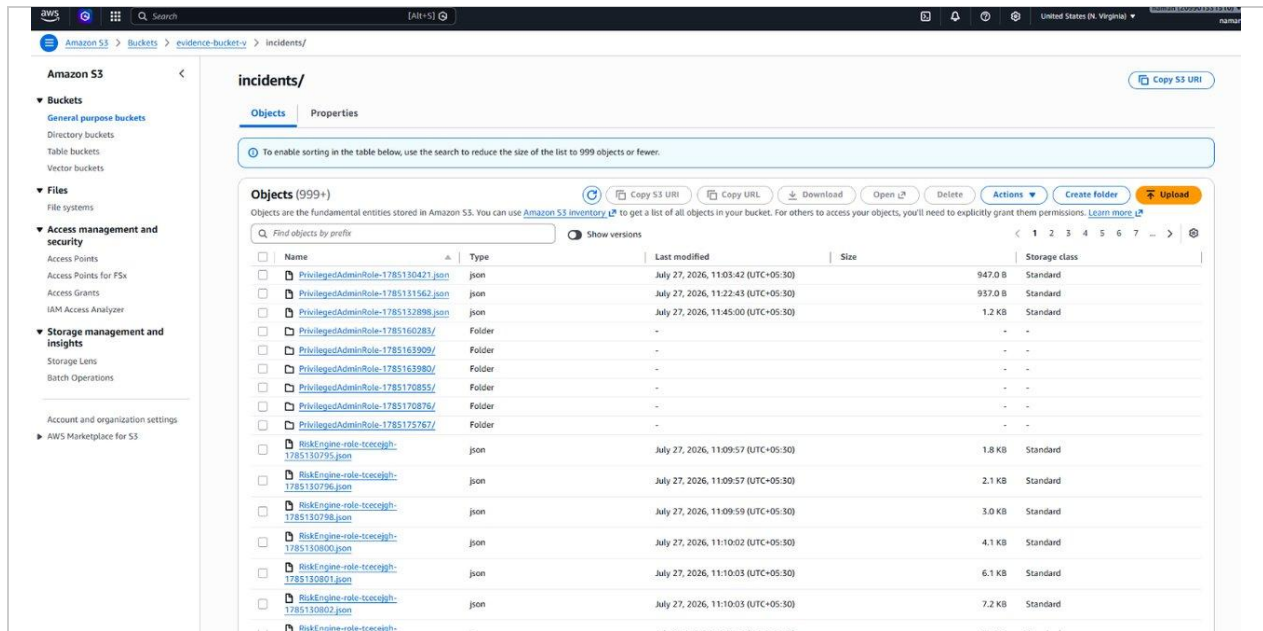


Figure 8.5 — Evidence bucket: one JSON object per incident, named by role and incident ID

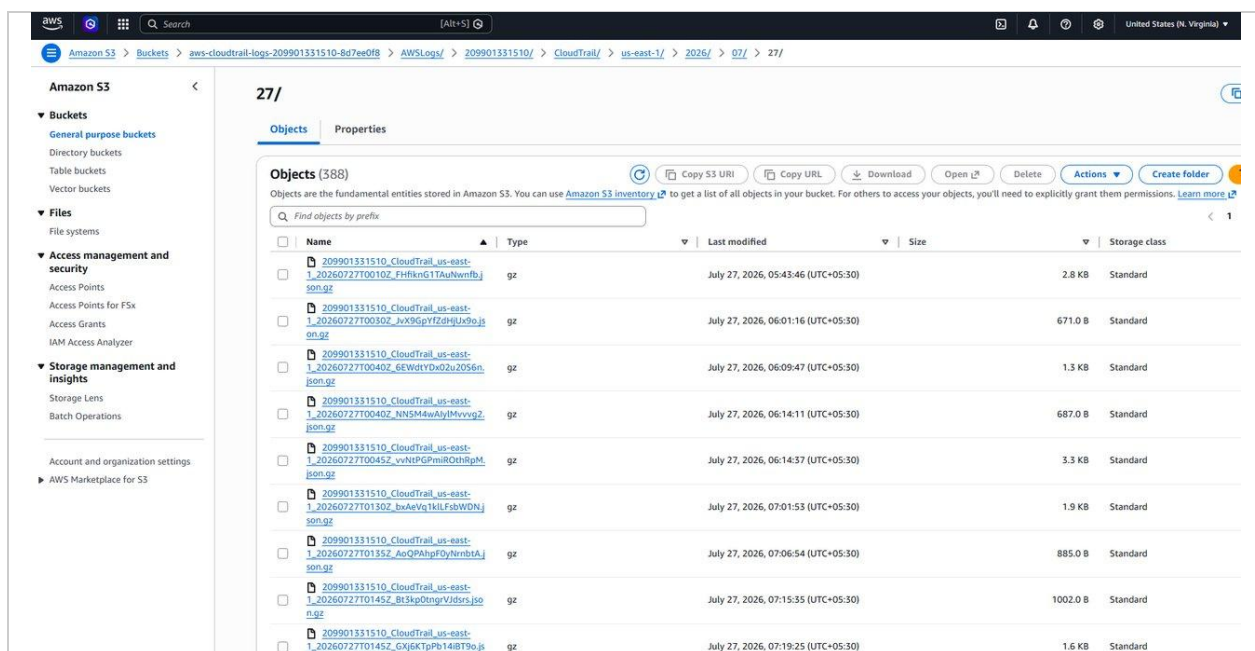


Figure 8.6 — Raw, gzip-compressed CloudTrail log objects preserved as the underlying factual record

9. Live Incident Walkthrough

The sequence below is a real, unscripted run of the system, captured screen-by-screen while operating the AWS console as the privileged role. It demonstrates the complete detect → decide → respond → notify loop end to end, using the exact scenario the risk engine is tuned for: a short burst of identity-management actions that ends in an account being deleted.

Step 1 — Operating inside the privileged session

With PrivilegedAdminRole assumed (visible in the top-right role badge), the operator navigates to IAM to begin the action sequence.

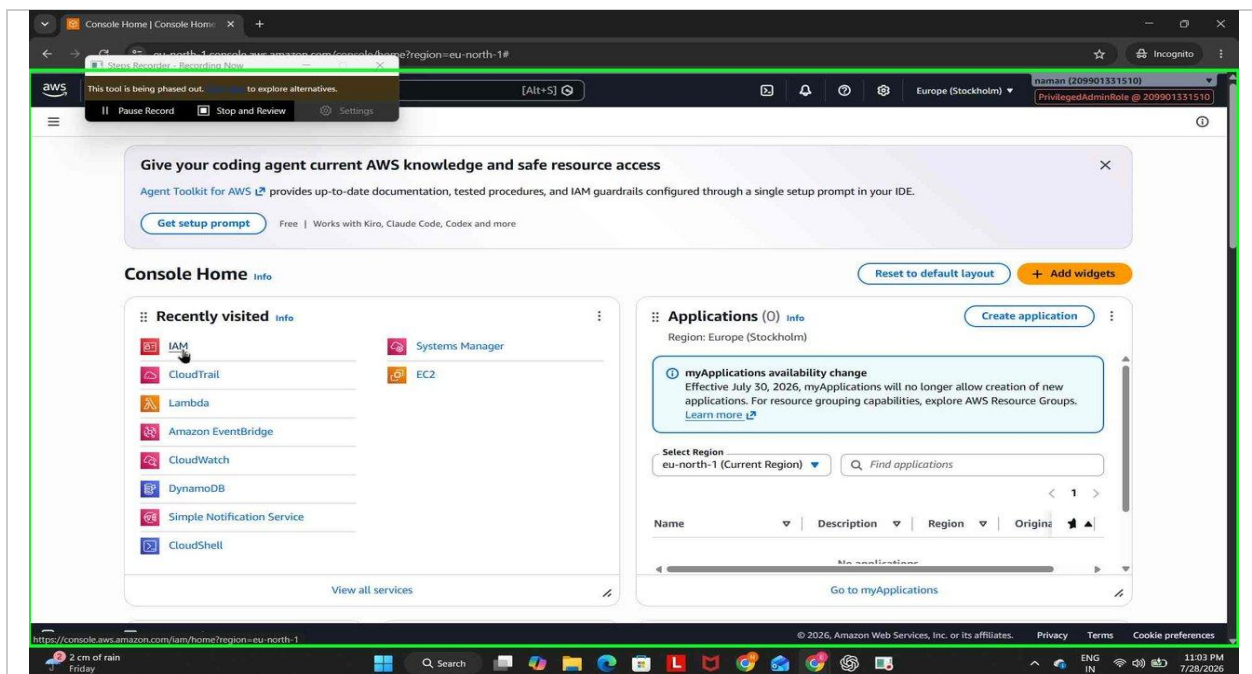


Figure 9.1 — Console Home, operating as PrivilegedAdminRole

Step 2 — Creating a new IAM user

A new user, naman, is created. On its own this is a mildly interesting, low-scored event.

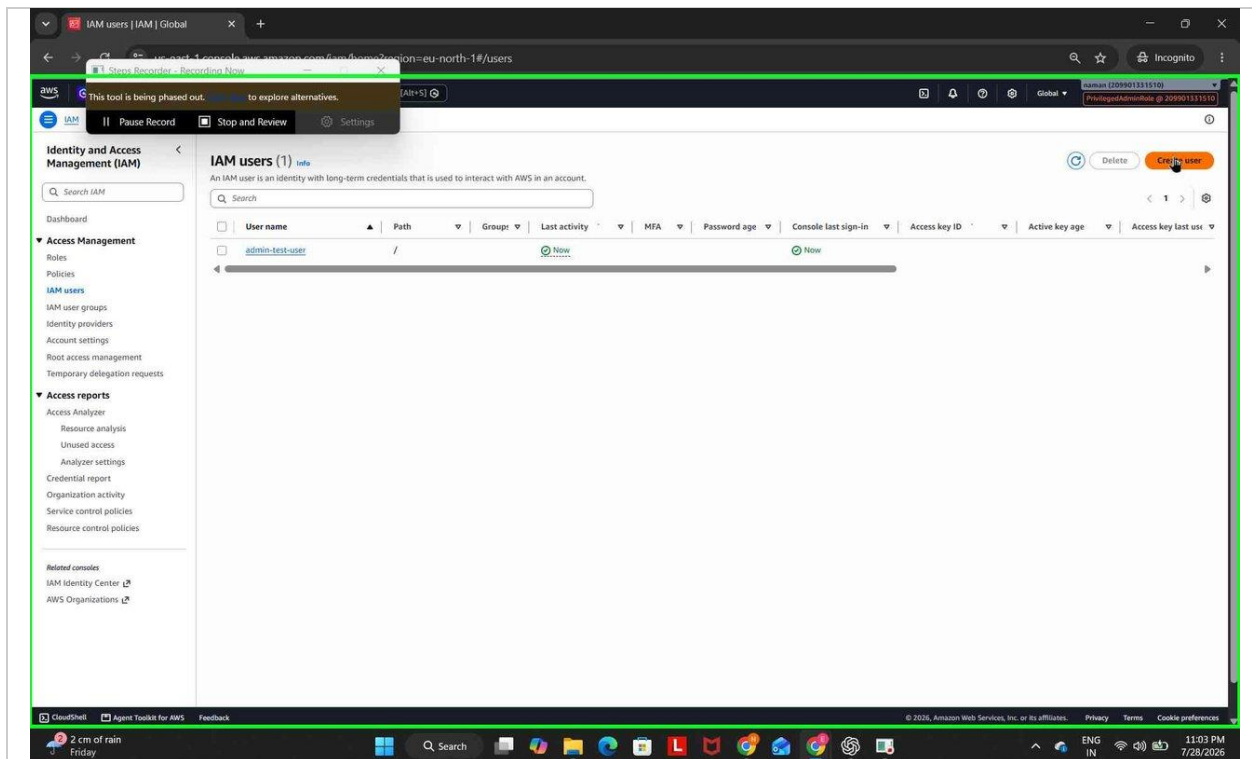


Figure 9.2 — IAM Users list before the new user is created

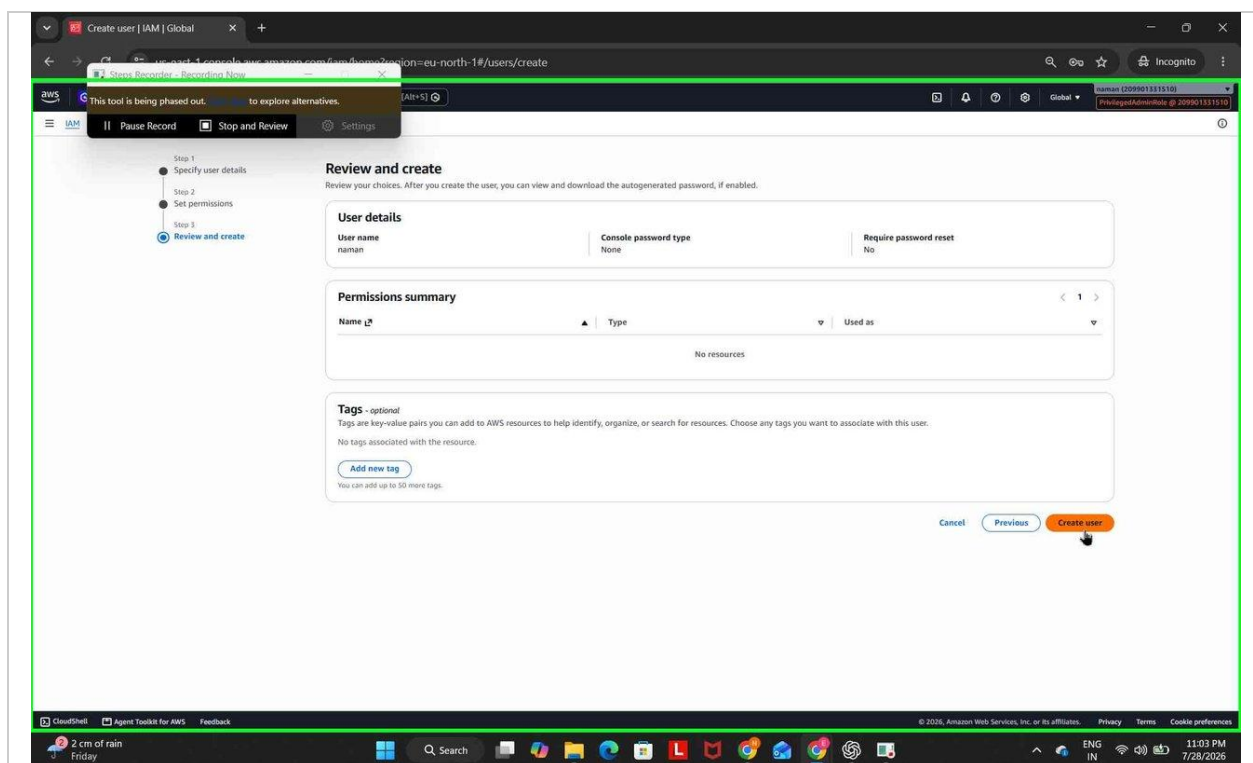


Figure 9.3 — “User created successfully” — the CreateUser event has now fired and reached the risk engine

Step 3 — Deleting the user minutes later

The same user is selected and deleted shortly after creation — a CreateUser → DeleteUser pattern inside a short window, which is exactly the kind of correlated behavior the 10-minute lookback is designed to catch.

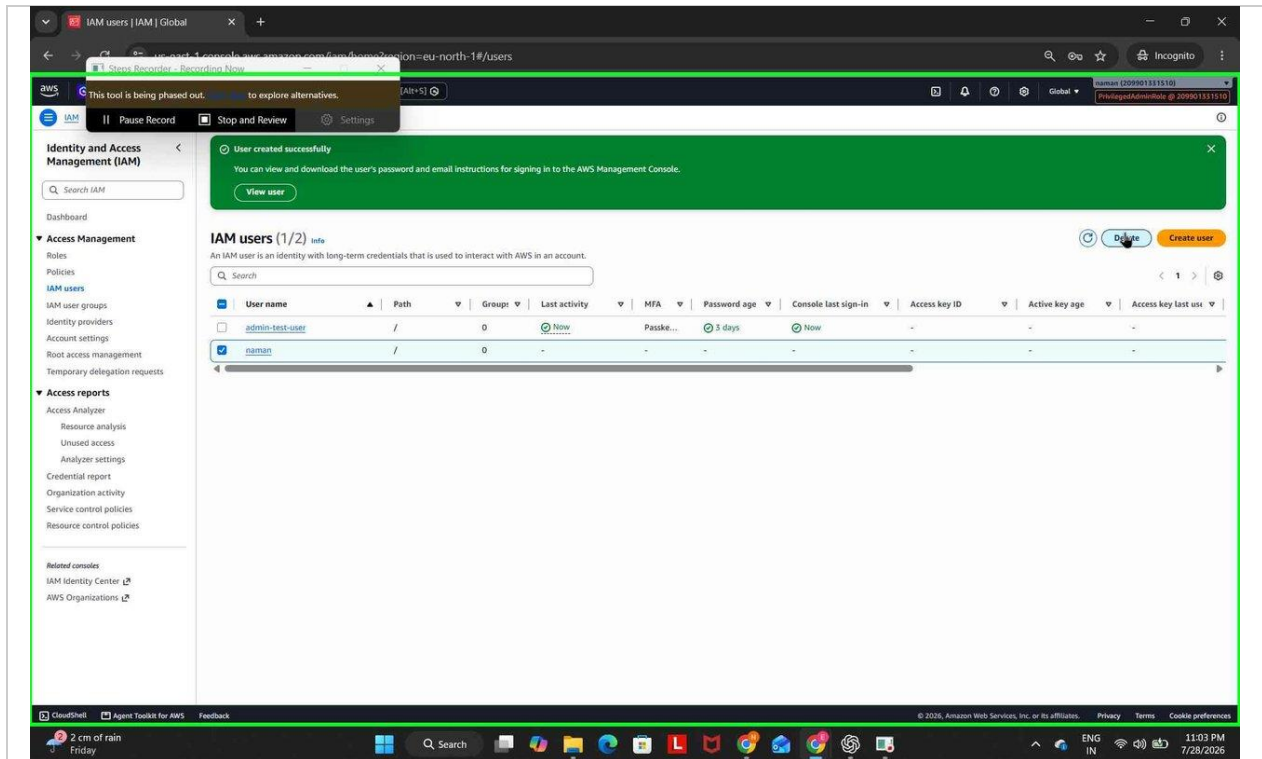


Figure 9.4 — Selecting the newly created user and choosing Delete

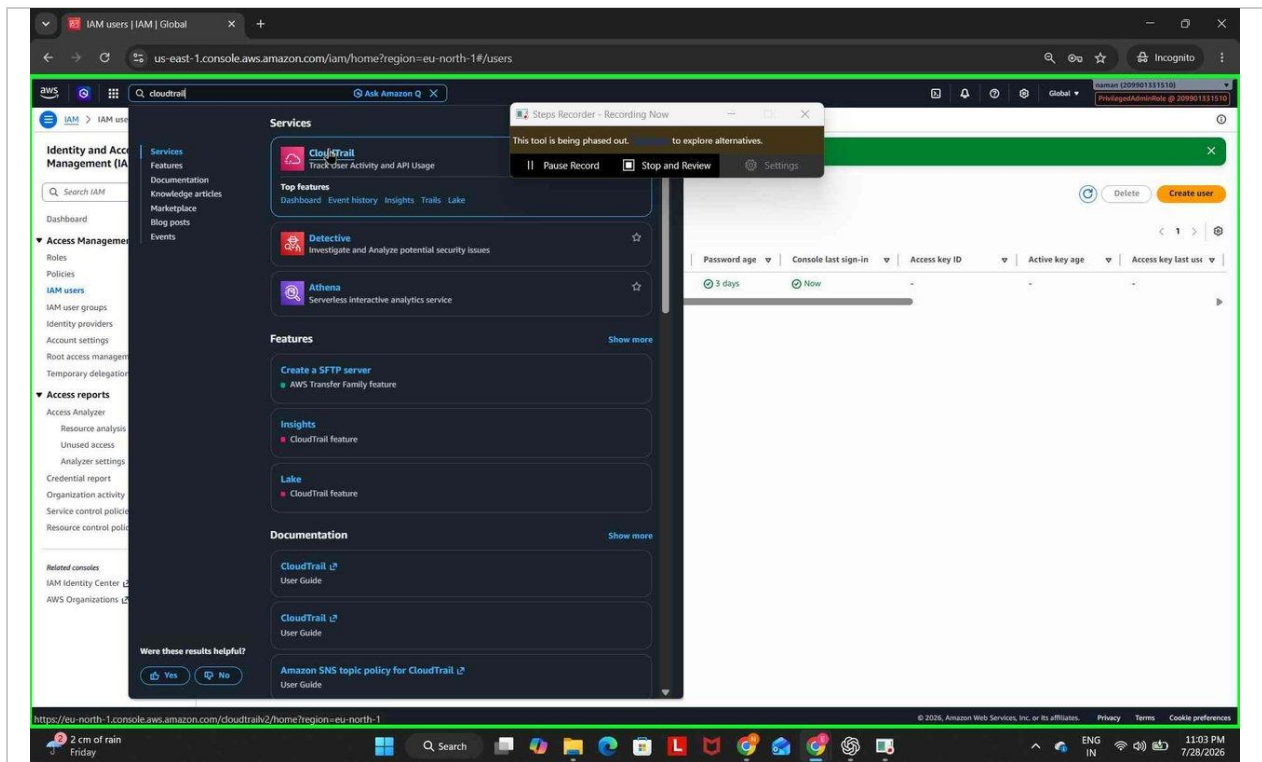


Figure 9.5 — User deleted; the operator now attempts to navigate to CloudTrail

Step 4 — Automated containment takes effect

Behind the scenes, EventBridge has already routed the DeleteUser event to RiskEngine, which correlated it with the recent CreateUser event, reached a cumulative risk score in the Critical range, and attached EmergencyDenyAllPolicy to PrivilegedAdminRole. The very next API call in the session — simply trying to open CloudTrail — is rejected:

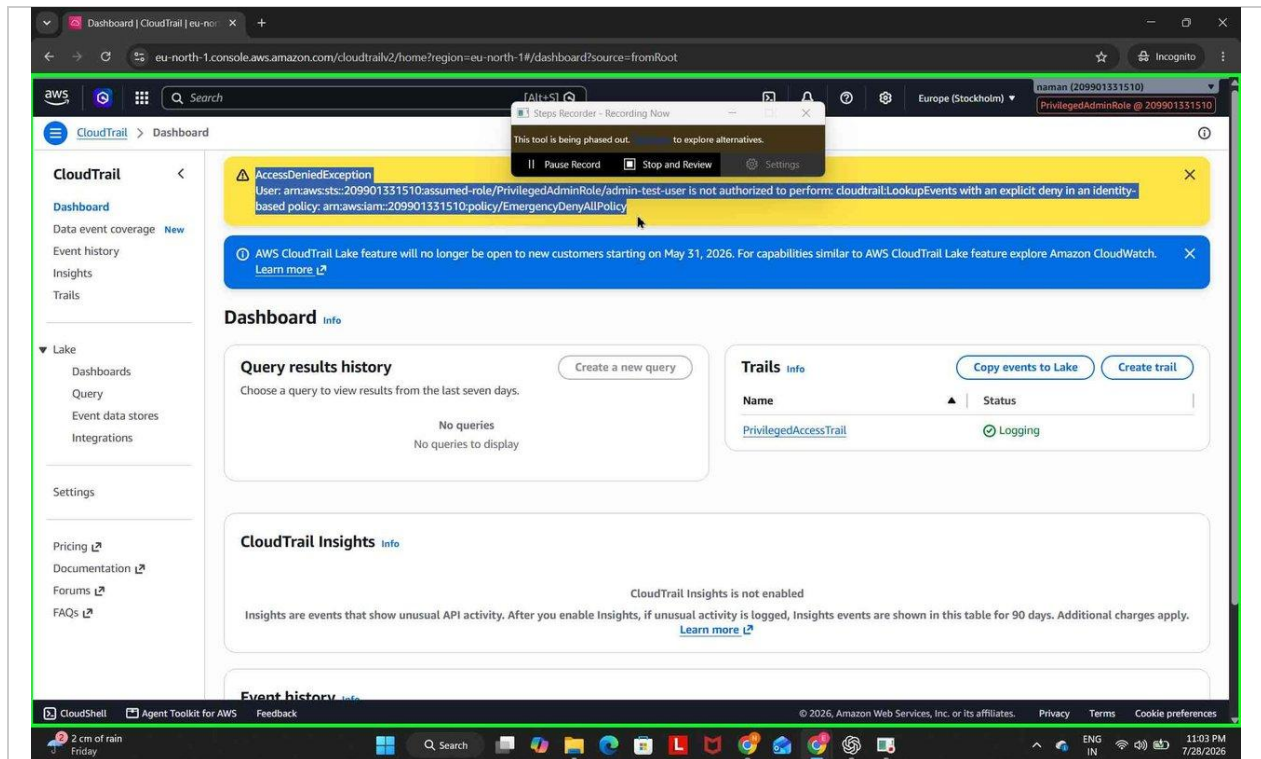


Figure 9.6 — AccessDeniedException: an explicit deny from EmergencyDenyAllPolicy blocks the still-active session within seconds of the Critical finding

What this proves

The session was not revoked by logging out or expiring — it was still authenticated. The deny took effect purely because the role's policy changed underneath it, confirming the “deny the role, not the user” design decision actually works against a live session, not just in theory.

Step 5 — The analyst is notified

At the same time, an SNS alert reaches the security team's inbox with no dashboard-watching required.

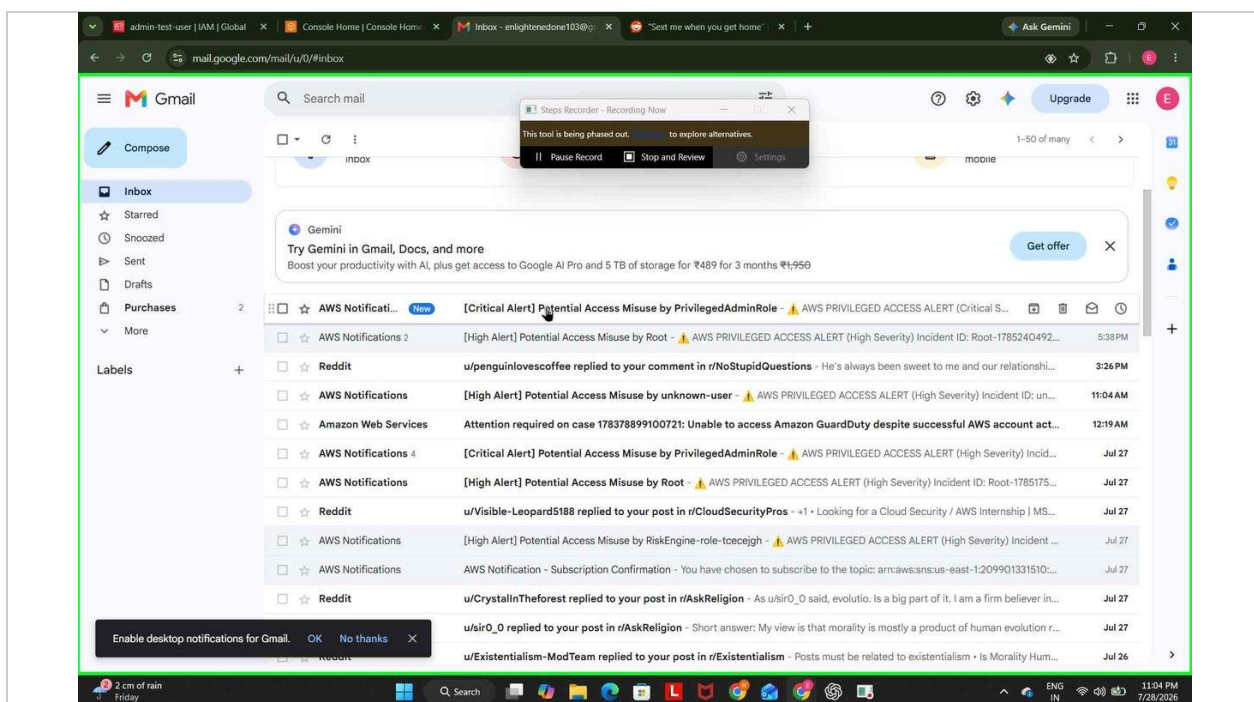


Figure 9.7 — Inbox showing the Critical-severity alert alongside earlier High-severity alerts from prior test runs

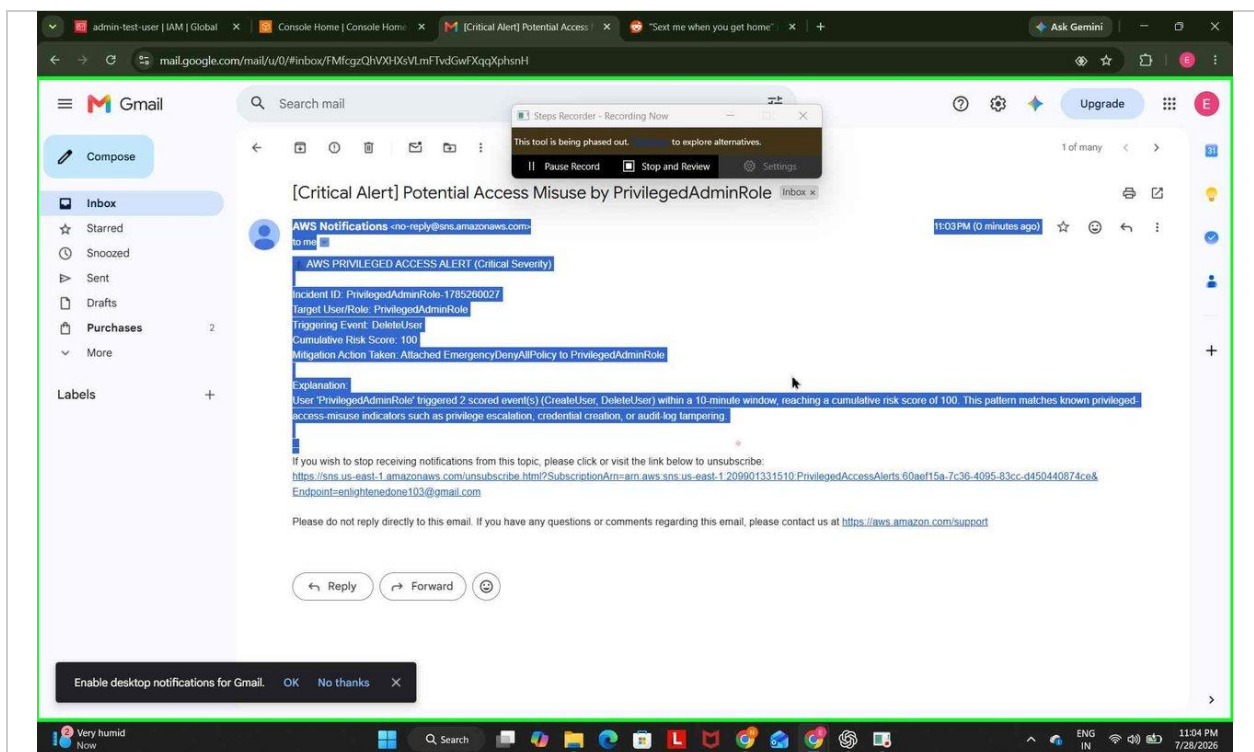


Figure 9.8 — Full alert content: Incident ID, target role, triggering event, cumulative score, mitigation action taken, and a plain-language explanation

The email is self-contained: an analyst can act on Incident ID PrivilegedAdminRole-1785260027 without opening the AWS console at all — it names the triggering event (DeleteUser), the cumulative score (100), the exact mitigation already applied (EmergencyDenyAllPolicy attached to PrivilegedAdminRole), and explains in one sentence why the pattern matched known privileged-access-misuse indicators.

Step 6 — A second, independent example: GuardDuty-triggered alert

Not every incident comes from IAM activity. This second alert was triggered purely by a GuardDuty finding on the root identity — confirming the GuardDuty rule path reaches the same risk engine and produces the same style of actionable notification.

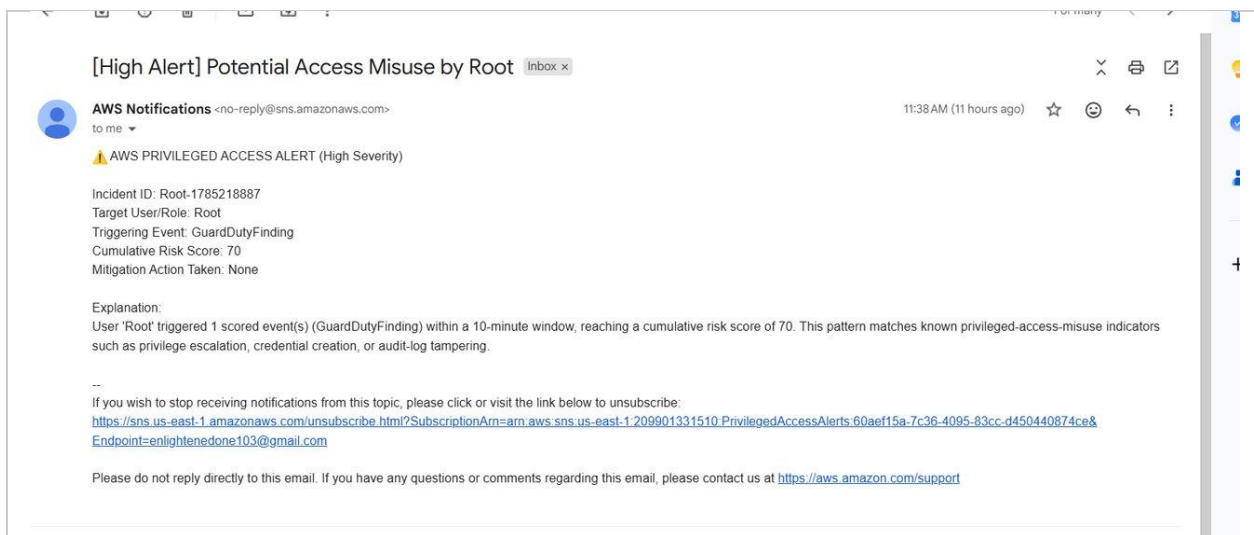


Figure 9.9 — High-severity alert triggered by a GuardDuty finding on the Root user, with the same structured format

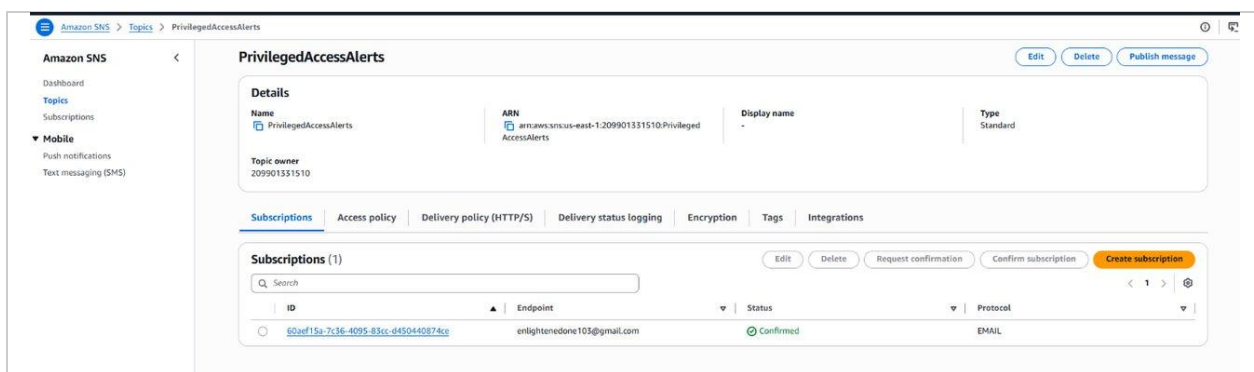


Figure 9.10 — The PrivilegedAccessAlerts SNS topic with a confirmed email subscription delivering these alerts

10. Observability

A CloudWatch dashboard shows recent incidents, invocation counts, and errors, so the system's health and activity are visible at a glance rather than requiring someone to dig through raw logs.

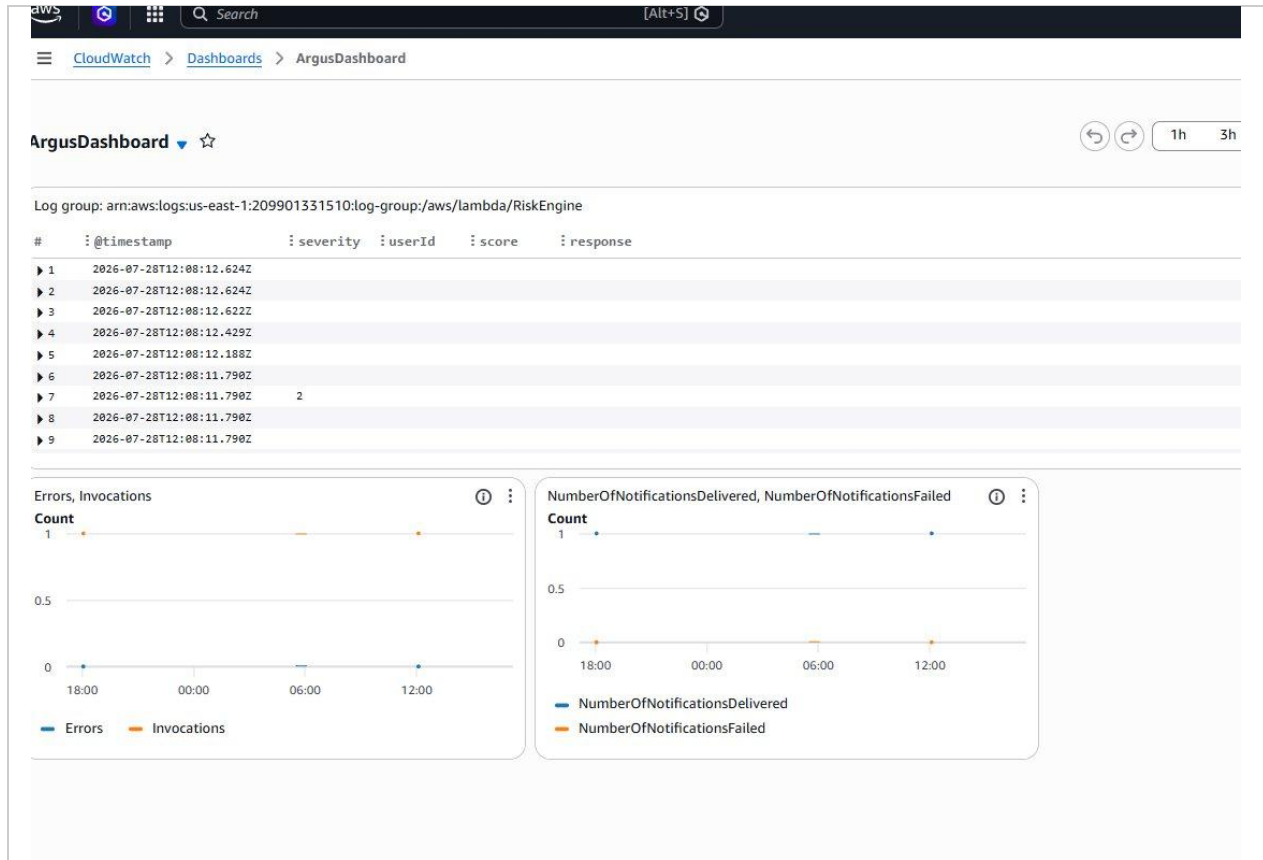


Figure 10.1 — ArgusDashboard: Lambda error/invocation counts and SNS delivery success

For deeper investigation, every risk-engine decision is traceable step by step in CloudWatch Logs — from the raw event received, through severity scoring, to the DynamoDB write, S3 evidence save, and SNS broadcast.

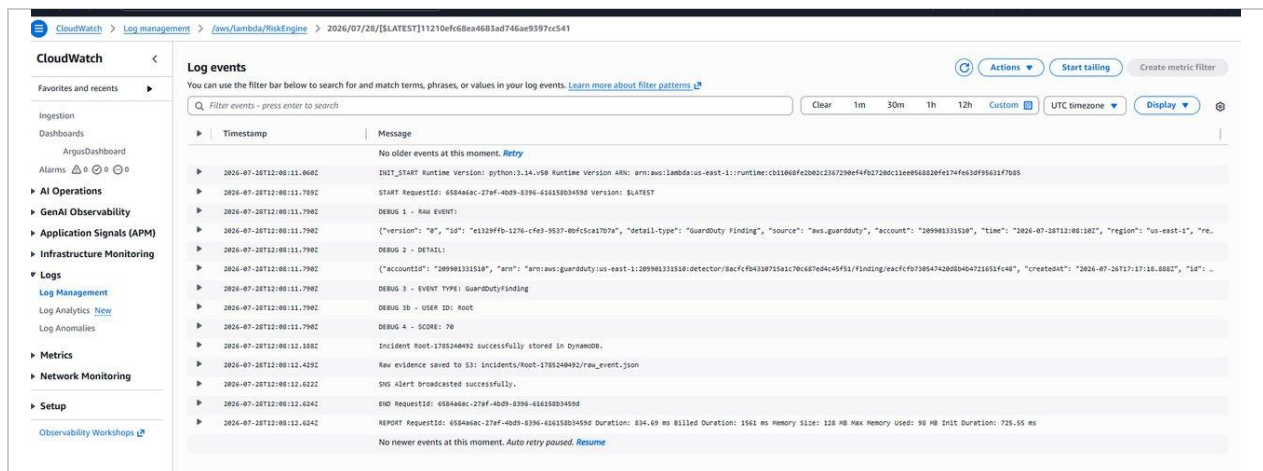


Figure 10.2 — A single RiskEngine invocation traced end to end: raw event → score 70 → incident stored → evidence saved → alert broadcast

11. Why It's Built the Way It Is

- Serverless throughout (Lambda, DynamoDB on-demand, EventBridge) — cost is incurred only when something actually happens, which matches how unpredictable privileged-access events actually are.
- Single region — IAM is a global AWS service, and its events always route through a specific region internally regardless of where you are working; building everything in that same region avoids an entire category of cross-region complexity.
- Deterministic explanation, not an external AI call — removes a dependency that can fail for reasons outside your control, while still delivering readable, specific explanations instead of raw numbers.
- Deny the role, not the user — the one decision most people get wrong, and the one most worth being able to explain precisely if asked (see Section 7.3 and the live proof in Section 9, Step 4).

12. Conclusion

Project Argus makes privileged power temporary by default, watches what that power is actually used for rather than trusting it blindly, correlates behavior over time instead of judging isolated events, explains what it finds in language a human can act on immediately, and shuts down access automatically the moment something crosses into clearly dangerous territory. The live walkthrough in Section 9 demonstrates that this is not just a design on paper: a real `CreateUser` → `DeleteUser` sequence, run through the actual deployed system, was detected, scored, contained, and reported — within seconds — turning what could be a slow-to-detect, high-damage insider incident into something caught, contained, and evidenced within minutes.